

DevOps Lab Manual

Lab 5 - Install Jenkins on an Ubuntu EC2 Instance

Objective

Install Java and Jenkins on an Ubuntu EC2 instance and access the Jenkins dashboard from a browser.

Prerequisites

- Ubuntu EC2 instance
 - SSH access
 - Security Group with:
 - SSH (22)
 - HTTP (80) (optional)
 - Custom TCP 8080 (required for Jenkins)
-

Step 1 - Update the Server

```
sudo apt update
sudo apt upgrade -y
```

Step 2 - Install Java

```
sudo apt install -y fontconfig openjdk-21-jre
```

Verify Java installation:

```
java -version
```

Troubleshooting

Scenario 1 - Java installation failed

Prompt:

OpenJDK installation failed on Ubuntu EC2. How do I fix it?

Scenario 2 - Permission denied

Prompt:

Permission denied while installing packages on Ubuntu EC2.

Step 3 - Install Jenkins

Add the Jenkins repository.

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

Add the repository.

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

Update packages.

```
sudo apt update
```

Install Jenkins.

```
sudo apt install -y jenkins
```

Step 4 - Verify Jenkins

```
sudo systemctl status jenkins
```

Enable Jenkins at boot.

```
sudo systemctl enable jenkins
```

Step 5 - Access Jenkins

Open:

```
http://<Public-IP>:8080
```

Example

```
http://13.233.xxx.xxx:8080
```

Note: Ensure port **8080** is allowed in the EC2 Security Group inbound rules.

Lab 6 - Change EC2 Instance Type

Objective

Resize an EC2 instance.

Steps

1. Stop the EC2 instance.
2. Go to

EC2 → Instance → Actions → Instance Settings → Change Instance Type

1. Change

t3.micro

to

t3.medium

Specifications

- 2 vCPUs
 - 4 GB RAM
-

Free Tier Note

If t3.medium is unavailable, choose one of:

- c7i-flex.large
 - m7i-flex.large
-

1. Save changes.
 2. Start the instance again.
-

Lab 7 - Initialize Jenkins

Step 1

Get the initial admin password.

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the password.

Step 2

Open

`http://<Public-IP>:8080`

Paste the password.

Step 3

Click

`Install Suggested Plugins`

Step 4

Create an administrator account.

Save the following safely:

- Username
- Password

These credentials will be required later.

Git and GitHub Introduction

Typical CI/CD Flow

```
GitHub Repository
  ↓
Jenkins Build
  ↓
Run Tests
  ↓
Docker Build
  ↓
Push Docker Image to Docker Hub
  ↓
Deploy to EC2 / Kubernetes / Terraform
```

Why Git?

Imagine five developers editing the same project.

Without Git:

- Code gets overwritten
- Conflicts occur
- Changes are lost
- Collaboration becomes difficult

Git enables teams to collaborate safely.

What is Git?

Git is a Version Control System that tracks changes in source code.

Real-world Analogy

Google Docs allows multiple people to:

- Edit simultaneously
- View history
- Restore previous versions

Git provides similar capabilities for source code.

Git Workflow

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Local Repository
  ↓
git push
  ↓
Remote Repository (GitHub)
```

Lab 8A - Create a Local Git Repository

Objective

Initialize a Git repository and make the first commit.

Create a project

```
mkdir my-project
cd my-project
```

Create files.

```
echo "This is File1" > file1.txt  
echo "This is File2" > file2.txt
```

Initialize Git.

```
git init
```

Check status.

```
git status
```

Stage files.

```
git add .
```

Commit.

```
git commit -m "Initial commit"
```

AI Prompts

- Explain git add vs git commit in simple terms.
 - Explain the Git workflow with an example.
-

Lab 8B - Connect Local Repository to GitHub

HTTPS Authentication

```
git remote add origin https://github.com/<username>/<repo>.git
```

Push.

```
git push -u origin main
```

GitHub may ask for a Personal Access Token (PAT).

SSH Authentication

Generate keys.

```
ssh-keygen -t rsa -b 4096
```

View public key.

```
cat ~/.ssh/id_rsa.pub
```

Add the key to GitHub.

Test connection.

```
ssh -T git@github.com
```

Update remote.

```
git remote remove origin
```

```
git remote add origin git@github.com:<username>/<repo>.git
```

Push.

```
git push -u origin main
```

Lab 8C - Branching and Merging

Create a branch.

```
git checkout -b feature1
```

Create another branch.

```
git checkout -b feature2
```

Switch branches.

```
git switch feature1
```

Merge.

```
git checkout main  
git merge feature1
```

Pull Request Workflow

```
Developer  
↓  
Feature Branch  
↓  
Push  
↓  
Pull Request  
↓  
Review  
↓  
Approve / Reject  
↓  
Merge
```

Lab 8D - Git Reset

Soft Reset

```
git reset --soft HEAD~1
```

Undo commit while keeping changes staged.

Mixed Reset

```
git reset --mixed HEAD~1
```

Undo commit and unstage files.

Hard Reset

```
git reset --hard HEAD~1
```

Deletes the last commit and all associated changes.

AI Prompt

Explain all Git reset modes with examples.

Lab 8E - Merge Conflicts

Create Conflict

On feature2

```
echo "Change from feature2" > file8.txt
```

Commit.

Switch to feature3

```
echo "Change from feature3" > file8.txt
```

Commit.

Merge both branches into main.

Git reports a merge conflict.

Resolve Conflict

Open the conflicted file.

Remove:

```
<<<<<<<
=====
>>>>>>>
```

Save.

Stage.

```
git add file8.txt
```

Commit.

```
git commit
```

AI Prompt

Resolve the following Git merge conflict.

Paste the error message into ChatGPT.

Lab 8F - Merge Strategies

Normal Merge

All commits remain.

```
A
 \
  B1
  B2
  B3
```

History contains all commits.

Squash Merge

```
A
 \
  B1
  B2
  B3
    ↓
Single Commit
```

History remains clean.

Rebase

Moves commits from one branch onto another to create a linear history.

Example:

```
git checkout feature1  
git rebase main
```

Practice Commands

Practice the following:

- git status
 - git log
 - git diff
 - git branch
 - git switch
 - git checkout
 - git merge
 - git pull
 - git push
 - git reset
 - git remote
 - git add
 - git commit
-

Suggested GenAI Prompts

1. Explain Git like I am a beginner.
2. Difference between Git and GitHub.
3. Explain Git branches with diagrams.
4. Explain Git merge vs rebase.
5. Explain Git reset with examples.
6. How does Git resolve merge conflicts?
7. Explain Git staging area with examples.

8. Best practices for Git commit messages.